Keys in the Relational Model

A **relation** is a table; each row is a **tuple** and each column is an **attribute**. Keys identify tuples and link tables.

- **Super key**: any set of attributes that uniquely identifies a tuple, for example {StudentID, Name}.
- **Candidate key**: a minimal super key; no attribute can be removed, for example {StudentID} or {Email}.
- **Primary key**: the candidate key chosen by the designer. It cannot be NULL and must be unique.
- **Alternate key**: a candidate key that was not chosen as the primary key.
- **Foreign key**: an attribute that refers to the primary key of another table, which enforces **referential integrity**.
- **Composite key**: a key made of two or more attributes, for example (StudentID, CourseID) in an Enrollment table.

Example

Student(StudentID, Name, Email, DeptID). StudentID is the primary key, Email is an alternate key, and DeptID is a foreign key referring to Department(DeptID).

> **Exam tip:** Every candidate key is a super key, but not every super key is a candidate key.

Entity Relationship Model

The **ER model** describes the database at the design stage using entities, attributes and relationships.

- **Entity**: a real-world object such as Student or Course. A **weak entity** has no key of its own and depends on an owner entity (for example Dependent of Employee).
- **Attributes**: simple, composite (Name = First + Last), multivalued (Phone numbers) and derived (Age from Date of Birth).
- **Relationship**: an association between entities, for example Student *enrolls in* Course.

Cardinality

Type	Example	How it becomes tables
One-to-one	Person – Passport	Foreign key on either side
One-to-many	Department – Employee	Foreign key on the 'many' side
Many-to-many	Student – Course	A separate table with both keys

Participation is **total** when every entity must take part in the relationship (every loan must belong to a customer) and **partial** otherwise.

> **Exam tip:** A many-to-many relationship always needs its own table, such as Enrollment(StudentID, CourseID).

Normalization: 1NF, 2NF, 3NF and BCNF

Normalization organises tables to remove redundancy and avoid **insertion, update and deletion anomalies**. It is based on **functional dependencies**: X → Y means the value of X decides the value of Y.

- **1NF**: every attribute holds atomic (single) values; no repeating groups.
- **2NF**: 1NF and no **partial dependency** (no non-key attribute depends on only part of a composite key).
- **3NF**: 2NF and no **transitive dependency** (no non-key attribute depends on another non-key attribute).
- **BCNF**: for every dependency X → Y, X must be a super key. BCNF is stricter than 3NF.

Worked example

Enrollment(StudentID, StudentName, CourseID, CourseName, Instructor, Grade) with key (StudentID, CourseID). Dependencies: StudentID → StudentName; CourseID → CourseName, Instructor; (StudentID, CourseID) → Grade.

StudentName and CourseName depend on part of the key, so the table is **not in 2NF**. Decompose into: **Student**(StudentID, StudentName), **Course**(CourseID, CourseName, Instructor) and **Result**(StudentID, CourseID, Grade). If the Course table also stored InstructorPhone with Instructor → InstructorPhone, that is a transitive dependency, so move it to **Instructor**(Instructor, InstructorPhone) to reach 3NF.

> **Exam tip:** Normalization should keep the decomposition lossless (no information lost) and, ideally, dependency preserving.

SQL Joins

A **join** combines rows from two tables using a related column.

Employee		Department	
EmpID, Name	DeptID	DeptID	DeptName
1, Asha	10	10	Sales
2, Ravi	20	30	HR
3, Meena	NULL		

- **INNER JOIN**: only matching rows. Result: Asha – Sales.
- **LEFT JOIN**: all employees, with NULL where there is no match. Result: Asha – Sales, Ravi – NULL, Meena – NULL.
- **RIGHT JOIN**: all departments. Result: Asha – Sales, NULL – HR.
- **FULL OUTER JOIN**: all rows from both tables.
- **CROSS JOIN**: every combination (3 × 2 = 6 rows).
- **SELF JOIN**: a table joined with itself, for example to find each employee's manager.

Example query: SELECT e.Name, d.DeptName FROM Employee e LEFT JOIN Department d ON e.DeptID = d.DeptID;

WHERE and HAVING

WHERE filters rows before grouping; HAVING filters groups after GROUP BY. Example: SELECT DeptID, COUNT(*) FROM Employee GROUP BY DeptID HAVING COUNT(*) > 5;

Exam tip: To find rows with no match (for example employees without a department), use a LEFT JOIN with WHERE d.DeptID IS NULL.

Transactions and ACID Properties

A **transaction** is a group of operations that form one logical unit of work, for example transferring ₹500 from account A to account B: read A, A = A − 500, write A, read B, B = B + 500, write B.

- **Atomicity**: all operations happen or none do. If the system fails after debiting A, the debit is rolled back. Handled by the recovery system using logs.
- **Consistency**: the transaction moves the database from one valid state to another; here A + B stays the same.
- **Isolation**: concurrent transactions do not see each other's partial results. Handled by concurrency control.
- **Durability**: once committed, changes survive crashes. Handled by writing logs to stable storage.

Transaction states

Active → Partially committed → Committed, or Active → Failed → Aborted (rolled back). After an abort, the transaction can be restarted or killed.

Problems without isolation

- **Dirty read**: reading data written by a transaction that later aborts.
- **Lost update**: two transactions update the same item and one update is overwritten.
- **Unrepeatable read**: reading the same item twice gives different values.

Exam tip: Atomicity and durability are provided by recovery; isolation is provided by concurrency control.

Concurrency Control and Serializability

A **schedule** is the order in which operations of concurrent transactions run. A schedule is **serializable** if its result equals some serial (one-after-another) schedule.

Conflict serializability

Two operations **conflict** if they belong to different transactions, access the same item, and at least one is a write (read–write, write–read or write–write). Build a **precedence graph** with an edge Ti → Tj for each conflict where Ti's operation comes first. The schedule is conflict serializable **if and only if the graph has no cycle**.

Example: R1(A), W2(A), W1(A). The conflicts give T1 → T2 (R1(A) before W2(A)) and T2 → T1 (W2(A) before W1(A)). The graph has a cycle, so the schedule is **not conflict serializable**.

Two-Phase Locking (2PL)

Shared (S) locks are for reading and exclusive (X) locks for writing. In 2PL each transaction has a **growing phase** (only acquires locks) and a **shrinking phase** (only releases locks). 2PL guarantees conflict serializability but can cause deadlocks. **Strict 2PL** holds exclusive locks until commit, which also avoids cascading rollbacks.

Timestamp ordering

Each transaction gets a timestamp; conflicting operations must follow timestamp order, otherwise the transaction is rolled back. No locks are used, so there are no deadlocks.

> **Exam tip:** Read–read is the only pair of operations that never conflicts.

Indexing and B+ Trees

An **index** is a data structure that speeds up searches on a column, like the index at the back of a book. It trades extra storage and slower inserts for much faster reads.

- **Primary (clustered) index**: the data file is sorted on the index key; only one per table.
- **Secondary (non-clustered) index**: a separate structure that points to rows; a table can have many.
- **Dense index**: an entry for every record. **Sparse index**: entries for only some records (one per block).

B+ tree

Most databases use a **B+ tree**: a balanced tree where all data pointers are in the **leaf nodes**, and the leaves are linked for fast **range queries**. Internal nodes only guide the search. Because the tree is balanced and has a high fan-out, a search takes O(log n) and needs only a few disk reads, even for millions of rows.

Point	B tree	B+ tree
Data pointers	In all nodes	Only in leaf nodes
Range queries	Slower	Fast (linked leaves)
Fan-out	Lower	Higher, so the tree is shorter

Hash indexes are very fast for equality searches (WHERE id = 5) but cannot answer range queries.

> **Exam tip:** Index the columns used often in WHERE, JOIN and ORDER BY, but avoid indexing columns that change very often.